

GUIA COMPLETO

WEB CRAWLER DO DICIONÁRIO FONÉTICO

Configuração, execução, arquitetura, depuração, geração de CSV e versionamento



Objetivo Documentar de forma prática e visual como configurar, executar, entender e versionar o crawler do Dicionário Fonético.

Versão organizada a partir do guia de configuração do projeto

Índice

Seção	Conteúdo
1	Estrutura do projeto
2	Tecnologias utilizadas
3-12	Configuração do ambiente
13-18	Funcionamento do crawler
19-23	Execução e arquivos CSV
24-30	Códigos de debug
31-39	Git e GitHub
40-41	Problemas comuns e interrupção
42-52	Atalhos e comandos principais
53	Fonte dos dados
54-55	Resumo e fluxo completo

Como usar este guia Os comandos aparecem em blocos visuais e podem ser copiados diretamente para o PowerShell. Diagramas resumem os fluxos mais importantes.

1. Estrutura do projeto

A estrutura atual do projeto é:

```
dicionario-fonetico/
├── .venv/
│   └── Ambiente virtual Python
├── data/
│   └── Dicionario_Fonetico_*.csv
├── debug/
│   ├── debug_01_acesso_pagina.py
│   ├── debug_02_estrutura_tabela.py
│   ├── debug_03_leitura_tds.py
│   ├── debug_04_extracao_registros.py
│   └── resultados/
│       └── arquivos gerados pelos testes de debug
├── .gitignore
├── GUIA_CONFIGURACAO.txt
├── main.py
├── README.md
└── requirements.txt
```

.venv/ — Ambiente virtual Python utilizado para instalar as dependências.

data/ — Pasta destinada aos dados coletados; os CSVs são gerados automaticamente.

debug/ — Códigos usados durante o desenvolvimento para investigar o site.

debug/resultados/ — Arquivos produzidos pelos testes de debug.

main.py — Crawler principal.

requirements.txt — Dependências externas.

README.md — Documentação resumida.

GUIA_CONFIGURACAO.txt — Procedimento completo de configuração e execução.

2. Tecnologias utilizadas

Python — Linguagem principal do projeto.

Requests — Realiza requisições HTTP ao Portal da Língua Portuguesa.

Pandas — Organiza os registros extraídos e gera os arquivos CSV.

Inquirer — Cria o menu interativo de seleção das regiões.

HTMLParser — Extrai conteúdo textual do HTML; faz parte da biblioteca padrão do Python.

Dependência padrão HTMLParser não precisa ser instalado separadamente.

3. Instalar o Python

No Windows, recomenda-se verificar a instalação com:

```
py --version  
python --version
```

O comando deverá apresentar uma versão do Python, por exemplo Python 3.x.x.

4. Abrir o projeto

Abra a pasta raiz `dicionario-fonetico/` no Visual Studio Code. Não abra somente o arquivo `main.py`.

```
File → Open Folder  
Selecione: dicionario-fonetico
```

5. Abrir o terminal

Use `Ctrl + `` ou `Terminal → New Terminal`. O terminal deve estar na pasta do projeto.

```
dir
```

Devem aparecer `main.py`, `requirements.txt`, `README.md`, `data` e `debug`.

6. Criar o ambiente virtual

```
py -m venv .venv
```

O ambiente virtual separa as bibliotecas do projeto das bibliotecas globais do computador.

7. Ativar o ambiente virtual

```
.\.venv\Scripts\Activate.ps1
```

Sucesso O terminal deverá mostrar algo como `(.venv) PS C:\...\dicionario-fonetico>`

8. Problema com o PowerShell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser  
.\.venv\Scripts\Activate.ps1
```

9. Verificar o ambiente virtual

```
python --version  
where.exe python  
python -m pip --version
```

O Python apontado por `where.exe python` deve estar dentro de `dicionario-fonetico\.venv\`.

10. Atualizar o pip

```
python -m pip install --upgrade pip
```

11. Instalar as dependências

```
python -m pip install -r requirements.txt
```

```
python -m pip list
```

12. Verificar as dependências

```
python -c "import requests, pandas, inquirer; print('Todas as dependencias foram instaladas corretamente!')"
```

13. Como o Web Crawler funciona

O crawler acessa o Dicionário Fonético do Portal da Língua Portuguesa e identifica a tabela rolloverable. Cada registro contém Palavra, Categoria Gramatical e Fonética.

Palavra	Categoria Gramatical	Fonética
a	artigo	a
a	preposição	a
á-bê-cê	nome masculino	,a.b,e.s'e
a-pro·pó·si·to	nome masculino	a.pɾo.p'ɔ.zi.tʊ

14. Paginação

A paginação utiliza o parâmetro start. Normalmente cada página possui até 20 registros.

```
start=0
start=20
start=40
start=60
...
```

Quando uma página possui menos de 20 registros, o crawler considera que chegou à última página daquela letra.

15. Letras

O crawler percorre automaticamente todas as letras do alfabeto:

```
a b c d e f g h i j k l m
n o p q r s t u v w x y z
```

16. Regiões disponíveis

Região	Código
Luanda	lda
Rio de Janeiro (não padrão)	rjo
Rio de Janeiro (padrão)	rjx
São Paulo (padrão)	spx
São Paulo (não padrão)	spo
Maputo (não padrão)	map
Maputo (padrão)	mpx
Lisboa (padrão)	lbx
Lisboa (não padrão)	lbn
Díli	dli

17. Menu de regiões

Ao executar `python main.py`, o programa apresenta um menu interativo. Use Espaço para marcar uma ou mais regiões e Enter para confirmar.

18. Teste inicial

Antes de uma coleta completa, recomenda-se selecionar somente uma região, como Luanda, para validar o ambiente e o crawler.

19. Executar o crawler

```
python main.py
```

O programa apresenta o título do crawler e, em seguida, o menu de regiões.

20. Informações exibidas no terminal

O crawler mostra informações resumidas, sem imprimir todas as palavras:

```
A (start=0): 20 palavras
A (start=20): 20 palavras
A (start=40): 20 palavras
...
→ Total A: 7263 palavras
```

21. Arquivos CSV

Os dados são armazenados em data/, usando o código da região.

```
data/Dicionario_Fonetico_lda.csv
data/Dicionario_Fonetico_lbx.csv
data/Dicionario_Fonetico_rjx.csv
```

22. Estrutura do CSV

Palavra	Categoria Gramatical	Fonética
a	artigo	a
a	preposição	a
á-bê-cê	nome masculino	,a.b,e.s'e
a-pro·pó·si-to	nome masculino	a.pro.p'ɔ.zi.tʊ

23. Codificação dos CSVs

```
encoding="utf-8-sig"
```

Essa codificação preserva acentos, caracteres especiais e símbolos do Alfabeto Fonético Internacional (IPA), além de facilitar a abertura no Excel.

24. Códigos de debug

A pasta debug/ contém códigos usados durante o desenvolvimento. Eles não são necessários para a execução normal do crawler.

```
debug/  
├─ debug_01_acesso_pagina.py  
├─ debug_02_estrutura_tabela.py  
├─ debug_03_leitura_tds.py  
└─ debug_04_extracao_registros.py
```

25. Debug 01 — Acesso à página

Verifica se o programa consegue acessar uma página, examinando status HTTP, URL final, tamanho da página e encoding. O HTML obtido é salvo em debug/resultados/.

Saída Os resultados dos testes são armazenados em debug/resultados/.

26. Debug 02 — Estrutura da tabela

Localiza a tabela rolloverable no HTML e extrai seu conteúdo para análise.

Saída Os resultados dos testes são armazenados em debug/resultados/.

27. Debug 03 — Leitura dos TDs

Analisa como os dados aparecem nas tags , observando a sequência Palavra → Categoria → Fonética.

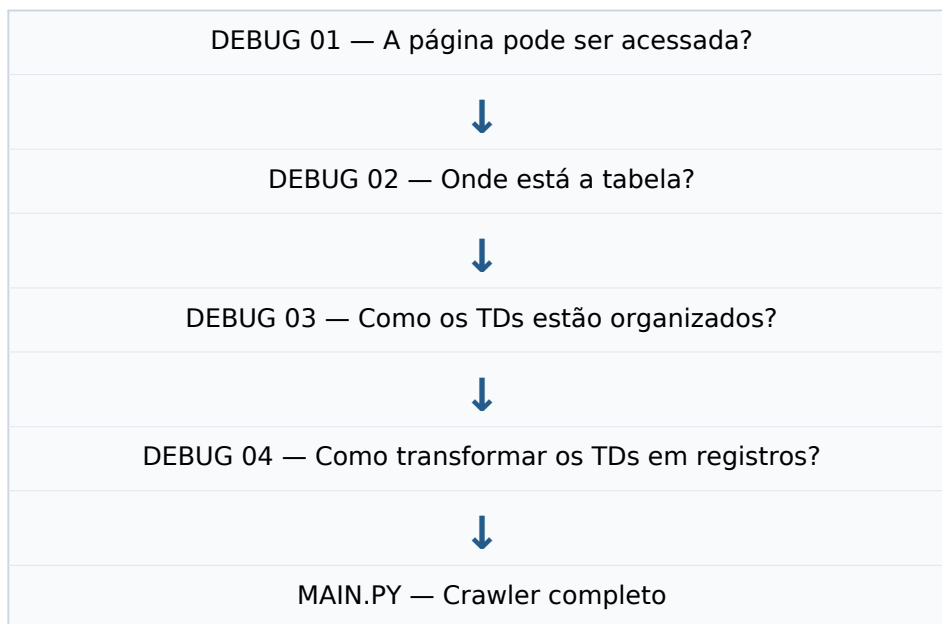
Saída Os resultados dos testes são armazenados em debug/resultados/.

28. Debug 04 — Extração dos registros

Valida a transformação dos dados em registros contendo Palavra, Categoria Gramatical e Fonética.

Saída Os resultados dos testes são armazenados em debug/resultados/.

29. Relação entre os debugs e o main.py



30. Executar os debugs

```
python debug\debug_01_acesso_pagina.py
python debug\debug_02_estrutura_tabela.py
python debug\debug_03_leitura_tds.py
python debug\debug_04_extracao_registros.py
```

31. Diferença entre debug e crawler

debug/ serve para investigação e validação. main.py é o programa principal de coleta.

32. Arquivos que devem ser versionados

```
.gitignore
main.py
requirements.txt
README.md
GUIA_CONFIGURACAO.txt
debug/
```

33. Arquivos que não devem ser versionados

```
data/
.venv/
debug/resultados/
```

34. Configuração do .gitignore

```
# Ambiente virtual
.venv/
venv/
env/
ENV/

# Python
__pycache__/
*.py[cod]

# Dados coletados
data/

# Resultados dos testes de debug
debug/resultados/

# VS Code
.vscode/

# IDEs
.idea/

# Arquivos temporários
*.log
*.tmp
*.temp

# Sistema operacional
.DS_Store
Thumbs.db
```

35. Inicializar o Git

```
git init
git status
git status --ignored
```

36. Verificar arquivos versionados

```
git ls-files
```

Os CSVs em data/ e os resultados em debug/resultados/ não devem aparecer.

37. Primeiro commit

```
git add .gitignore main.py requirements.txt README.md GUIA_CONFIGURACAO.txt debug/  
git status  
git commit -m "feat: adiciona crawler do dicionario fonetico"
```

38. Publicar no GitHub

```
git remote add origin https://github.com/SEU_USUARIO/dicionario-fonetico.git  
git remote -v  
git branch -M main  
git push -u origin main
```

Atenção Substitua SEU_USUARIO pelo seu usuário real do GitHub.

39. Estrutura esperada no GitHub

```
dicionario-fonetico/  
├── debug/  
│   ├── debug_01_acesso_pagina.py  
│   ├── debug_02_estrutura_tabela.py  
│   ├── debug_03_leitura_tds.py  
│   └── debug_04_extracao_registros.py  
├── .gitignore  
├── GUIA_CONFIGURACAO.txt  
├── README.md  
├── main.py  
└── requirements.txt
```

Não enviar .venv/, data/*.csv e debug/resultados/

40. Problemas comuns

1 — Activate.ps1 não encontrado

```
dir .env  
  
dir .env\Scripts  
  
Se necessário:  
Remove-Item -Recurse -Force .env  
py -m venv .env
```

2 — PowerShell bloqueia Activate.ps1

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser  
.\.env\Scripts\Activate.ps1
```

3 — Python incorreto

```
where.exe python  
  
O primeiro resultado deve estar dentro de .env\Scripts\.
```

4 — Biblioteca não encontrada

```
python -m pip install -r requirements.txt
```

5 — requirements.txt não encontrado

```
dir  
  
Você precisa estar na pasta que contém main.py e requirements.txt.
```

6 — CSV não aparece

```
Verifique se o programa terminou sem erro e execute:  
  
dir data
```

7 — Caracteres estranhos no CSV

```
O crawler utiliza encoding="utf-8-sig".
```

8 — O site não responde

```
Verifique conexão, disponibilidade do portal, endereço e resposta HTTP. Evite requisições simultâneas excessivas.
```

41. Interromper o crawler

```
Ctrl + C
```

O crawler atual não utiliza checkpoint. Uma nova execução da região começa novamente do início; não existe a pasta progress/ nem arquivos de checkpoint.

42. Abrir o terminal no VS Code

Ctrl + ` ou Terminal → New Terminal.

43. Abrir a paleta de comandos

Ctrl + Shift + P e procure Python: Select Interpreter.

44. Selecionar o Python do projeto

Selecione o interpretador localizado dentro de .venv.

45. Abrir o Explorer

Ctrl + Shift + E

46. Pesquisar no projeto

Ctrl + Shift + F

47. Pesquisar no arquivo atual

Ctrl + F

48. Salvar o arquivo

Ctrl + S

49. Executar o crawler

```
python main.py
```

50. Executar um debug

```
python debug\debug_01_acesso_pagina.py
```

51. Limpar o terminal

```
cls
```


52. Comandos principais

```
# Configuração
py --version

# Criar ambiente
py -m venv .venv

# Ativar
.\.venv\Scripts\Activate.ps1

# Verificar Python
python --version

# Verificar Python utilizado
where.exe python

# Verificar pip
python -m pip --version

# Atualizar pip
python -m pip install --upgrade pip

# Instalar dependências
python -m pip install -r requirements.txt

# Executar crawler
python main.py

# Executar debug
python debug\debug_01_acesso_pagina.py

# Git
git status
git status --ignored
```

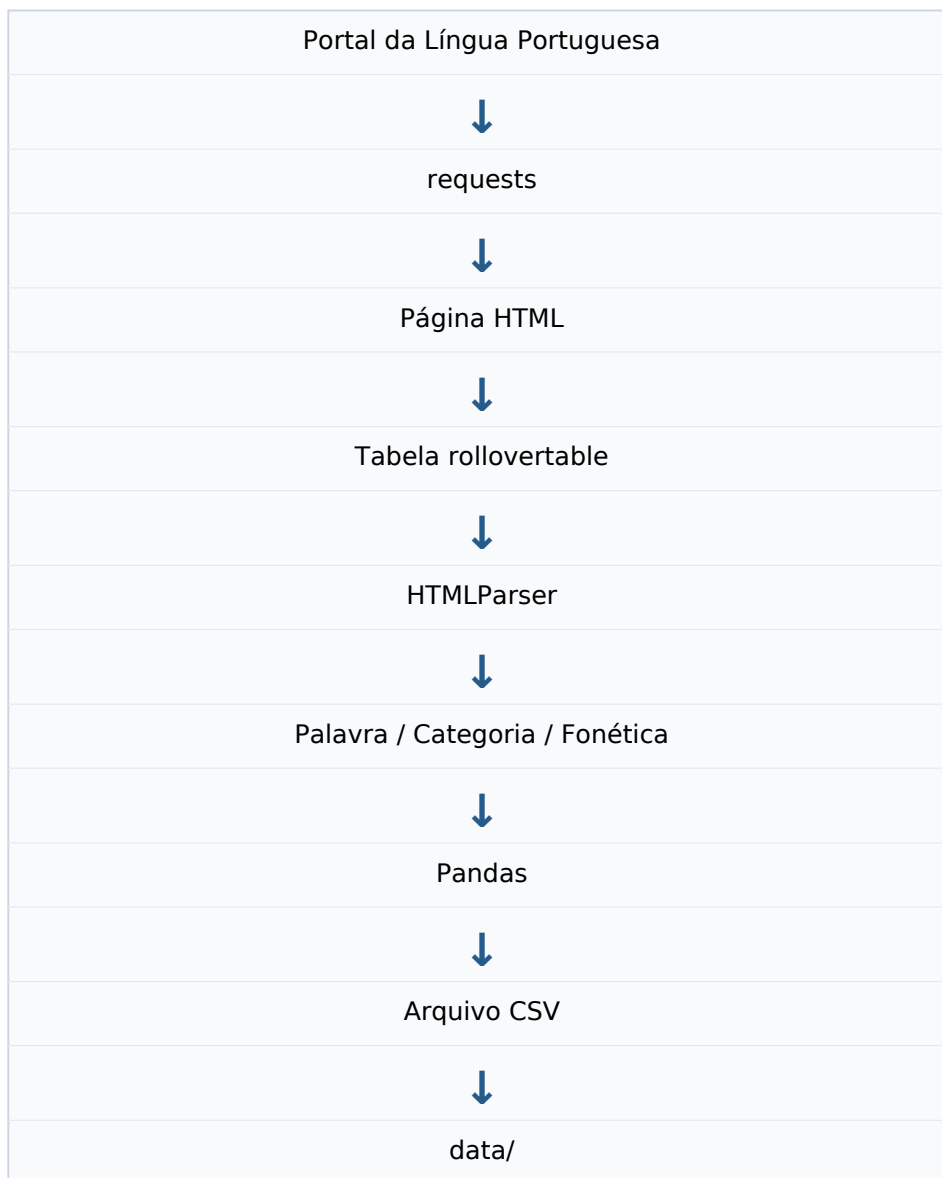
53. Fonte dos dados

O crawler coleta dados do Dicionário Fonético do Portal da Língua Portuguesa.

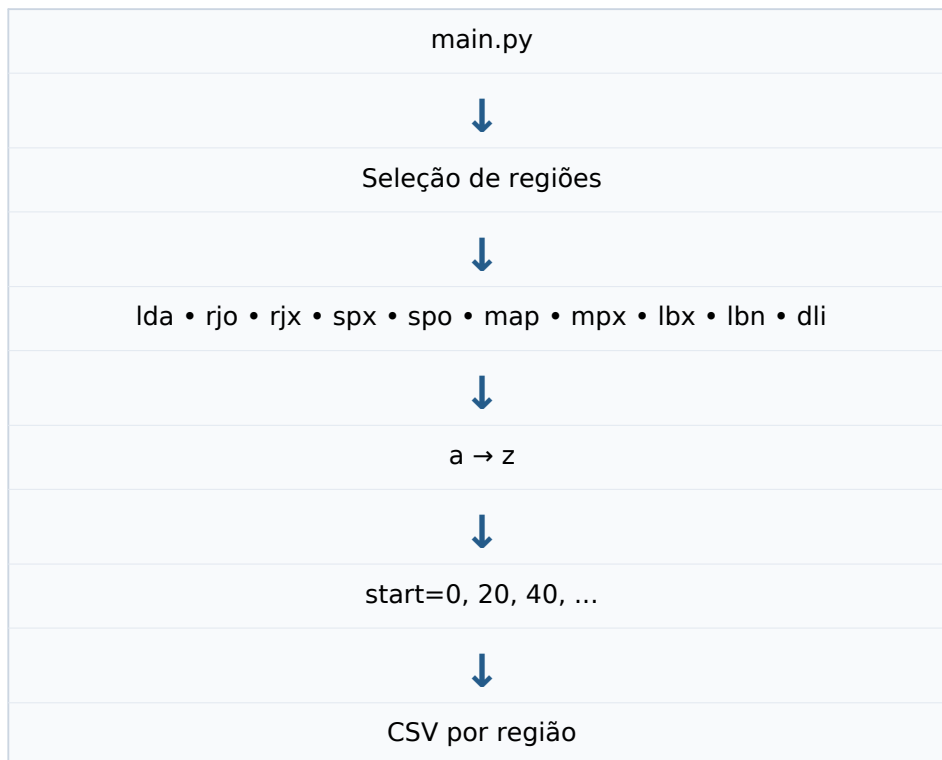
Fonte <https://www.portaldalinguaportuguesa.org/>

Os dados coletados são provenientes de uma fonte externa. Antes de redistribuir os dados coletados, verifique os termos de uso e as condições de acesso do portal.

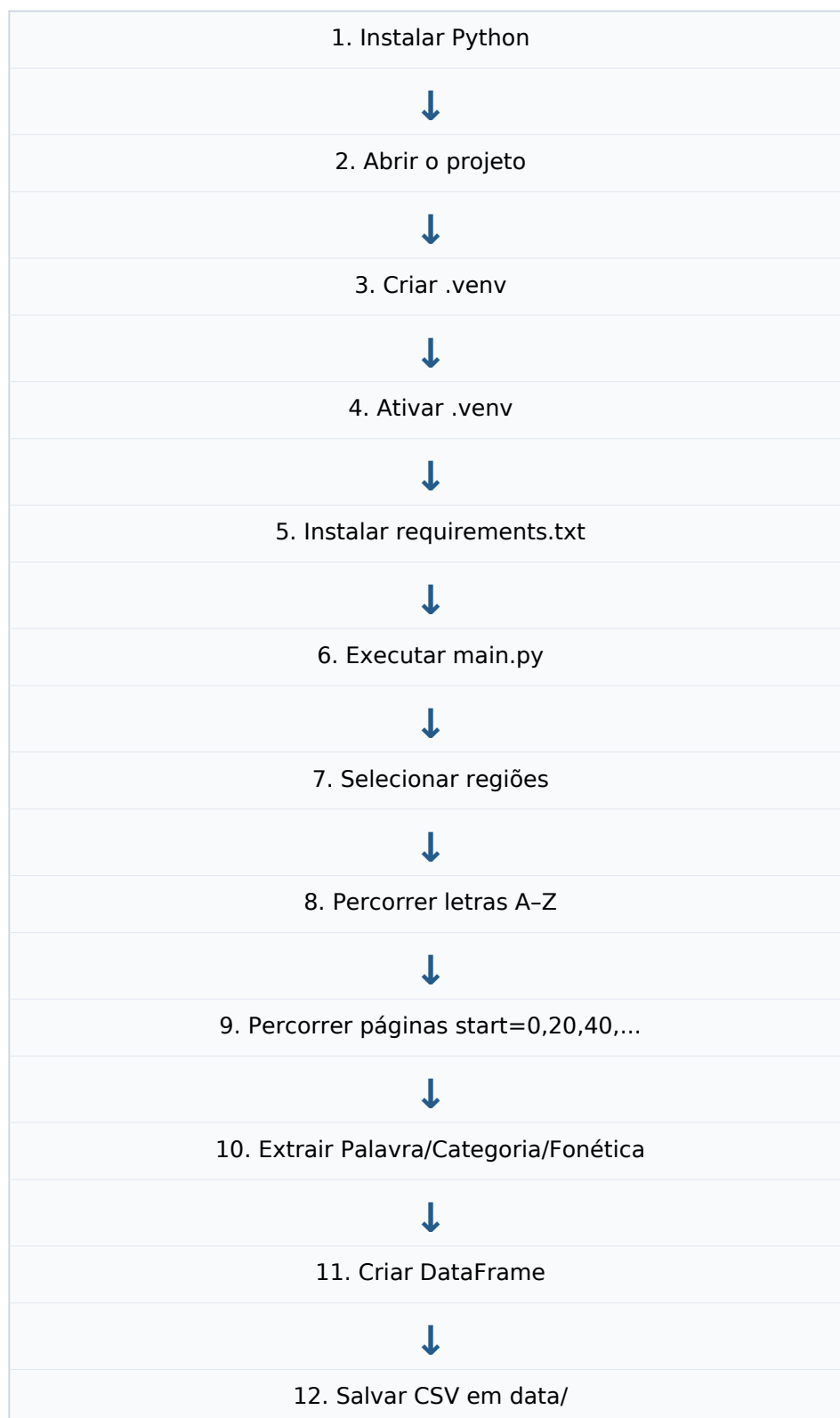
54. Resumo do funcionamento



Para múltiplas regiões:



55. Fluxo completo do projeto



Conclusão Os arquivos debug_01 até debug_04 documentam as etapas usadas para compreender a estrutura do site e desenvolver a extração. O main.py consolida esse conhecimento no crawler principal.

FIM